

O'REILLY®

Transforming and Analyzing Data With DuckDB

From zero to analytical power
in just 3 hours





Learning Objectives

By the end of this course, you will understand:

- How to integrate duckdb into your data analysis (with or without Python).
- How to work with different data formats and how to query or consolidate them.
- How to use duckdb for online analytical processing (OLAP).
- How to integrate duckdb with pandas and avoid bottlenecks.
- How to speed up your data-driven Jupyter notebooks.





Agenda

- Introduction
- What is DuckDB (30 minutes)
- Getting data in and out (30 minutes)
- SQL in DuckDB (40 minutes)
- DuckDB integration with Python (20 minutes)
- Performance (30 minutes)
- Real-life data analytics example (30 minutes)
- Summary & discussion



Introduction

- About me (Christian Winkler)
 - Programming for more than 40 years
 - PhD in physics, working as a professor at a university of applied science
- About the course
 - Working a lot with data
 - Different formats, large size, bottlenecks
 - DuckDB as a swiss-army knife **and** analytical engine





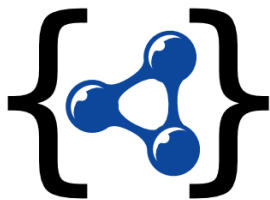
Introduction: data volume and file heterogeneity



Data challenges



{JSON}

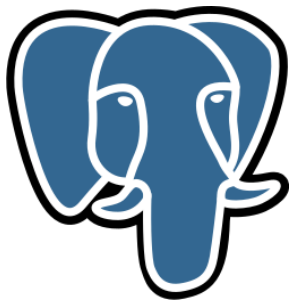
{
JSONL



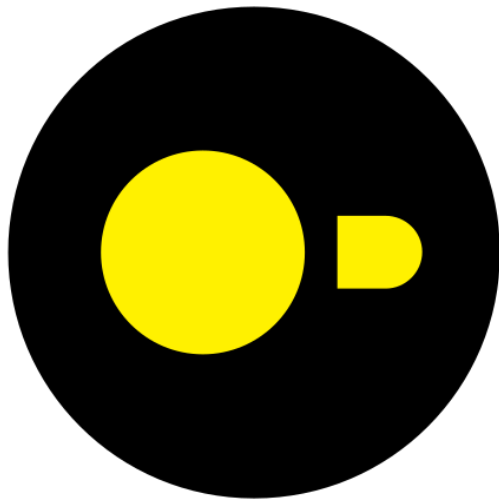
Hundreds of millions of rows
Thousands of columns

Your session crashed after using all available RAM. [View runtime logs](#) X

Possible solutions



Possible solutions



DuckDB



What is DuckDB?



Handling large amounts of data

Typical problem

- You have one (or many) large CSV or JSON file(s)
- The data is too large for pandas
- You are only interested in some aggregates

Typical solution

- Import the file(s) to a database
- Query the database

Challenge

- Databases need a server
- sqlite is an alternative, but slow
- You have to write (and maintain!) glue code

DuckDB on the stage

- Does not need a server
- Is super fast
- Can work with these file formats directly
- Does not need glue code



Key features of DuckDB

Runtime type

- In-process (does not need a server)
- Efficient memory usage (written in C++)
- Data can be kept in memory

Speed

- Vectorized execution engine (one CPU)
- Query planner (parallel execution)

Query language

- SQL (superset)

Data format

- Many formats supported (CSV, JSON, Parquet, many more via plugins)
- Own native dataformat
- Prefers Columnar data (unlike relational DBs, OLTP)

Integrations

- Python (pandas, polars)
- R



Quoting “DuckDB in Action”

It is **faster** than **SQLite** for analytical workloads.

It is **easier** to set up than a **Spark** cluster.

It has **lower resource requirements** than **pandas**.

It doesn't throw **weird Rust** errors like **Polars**.

It is **easier** to set up and use than **PostgreSQL**, **Redshift**, and other relational databases.

It is **faster and more powerful** for data transformations than **Talend**.



Installation of DuckDB

MacOS

- `brew install duckdb`

Windows

- `winget install DuckDB.cli`

Python or Google Colab

- `pip install duckdb duckdb-cli`
- `uv add duckdb duckdb-cli`

Running with in-memory mode

- `duckdb`
- Persistence can be connected later
- Files can be queried directly
- Files can also be written

Running with persistence

- `duckdb database.duckdb`
- Work with persistent tables like in other databases



Hands-on exercise:

Install DuckDB and run first query on CLI and in Python



Q&A





Poll

What is so special about DuckDB?

- It runs in memory
- It has a powerful server
- It's easy to embed in Python
- It's fast only for small amounts of data



Getting data in and out



Querying different file formats natively

CSV

- Query like a table
- Data types are inferred

Excel

- Can use sheets, ranges etc.

JSON

- Can also be queried like a table
- Data types more strict than in CSV

Parquet

- Columnar format for one table
- Very space efficient
- Excellent for aggregation
- Integrations for other software available

DuckDB files

- Internal database format
- Very efficient, multiple tables etc.

Extensions for additional file formats

XML

- Very helpful for HTML and XML
- Community extension [webbed](#)

Sitemaps

- Also supported via [community extension](#)
- Be careful, does not work (like a few!)

S3

- Use [httpfs extension](#)
- Also suitable for reading from S3 buckets

Database integration

- Query data from other databases
- [Core extensions](#) allow Iceberg, MySQL, ODBC, SQLite

DuckDB files

- Internal database format
- Very efficient, multiple tables etc.



Converting to file formats

Aggregate different files

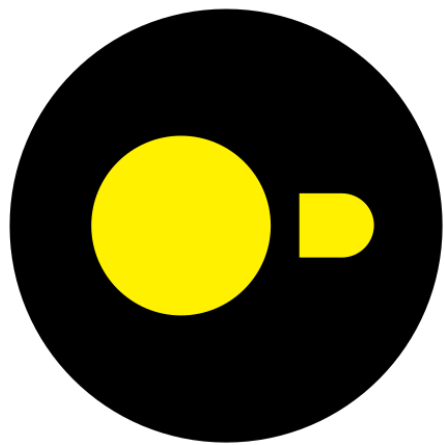
- SELECT also allows wildcards
- Very good if you have multiple files
- Saves enormous amount of repetitive programming effort

Convert to new formats

- Choose most efficient format
- Consider interoperability

Convert to parquet

- Most often a very good decision
- Very space-efficient
- Columnar storage allows fast aggregation
- Can be read in pandas, polars etc. (also just some rows)
- Not good if you need many (or all) columns
- Avoid if you need to update (not possible, create multiple files then)



DuckDB



Output options

Standard output

- Nice box view (duckbox)
- Very good for reading
- Not so good for pasting

Many other output formats are available

- Most popular/useful: markdown, csv, tabs, json, duckbox

Can use duckdb as a script

- Options -f or -c

Date date	Location varchar	MinTemp double
2008-12-01	Albury	13.4
2008-12-02	Albury	7.4
2008-12-03	Albury	12.9
2008-12-04	Albury	9.2
2008-12-05	Albury	17.5
2008-12-06	Albury	14.6
2008-12-07	Albury	14.3
2008-12-08	Albury	7.7
2008-12-09	Albury	9.7
2008-12-10	Albury	13.1

10 rows

3 columns



Hands-on exercise:
Query multiple JSON files,
transform to Parquet format, and
query Parquet



Q&A





Poll

Which data formats can be used natively in DuckDB?

- Parquet
- Excel
- CSV
- JSON



SQL in DuckDB



SQL refresh

SQL

- Structured query language
- Standard for relational databases

SELECT statement

- SELECT column1, column2
FROM table
WHERE condition
GROUP BY column1
HAVING aggregate_condition
ORDER BY column1 DESC
LIMIT 10;

Works like a pipeline

- SELECT → What do I want?
- FROM → Where is it?
- WHERE → Which rows?
- GROUP BY → How should rows be aggregated/grouped?
- HAVING → Filter after aggregation?
- ORDER BY → In what order?
- LIMIT → How many results?



Tables, views, indices

Tables

- More or less like in normal SQL
- CREATE OR REPLACE syntax
- TEMPORARY TABLE

Views

- Just as a shortcut
- No materialized views
- Better use temporary tables for that

Indices

- min-max index (zonemap):
automatically created (!)
- Adaptive Radix Tree: useful for specific columns (UNIQUE, PRIMARY KEY)
- Spatial indices supported by
`spatial` extension
- Vector type (for RAG etc.) using HNSW
available via [vss extension](#)



Data types

Traditional data types

- String (varchar),
- Numeric (integer, float, and decimal in different sizes),
- Dates (timestamps, intervals)
- Boolean
- Binary large objects (BLOBs).

Structured data types

- Enums: indexed, named elements
- Lists (arrays): multiple elements of the same type, including functions for this
- Maps or dicts: key-value pairs
- Structs: structured key-value objects, same keys have same data types (inferred, used for JSON)



Common Table Expressions (CTE)

Definition

- A common table expression is a temporary result set that you can use like a table

Syntax

- `WITH cte AS (SELECT * FROM table)`
`SELECT * FROM cte;`

Useful

- Easier reading of statements
- Avoid repetition

Example

```
WITH temp_rain AS
(SELECT AVG(MinTemp) AS avgMin,
  AVG(MinTemp) FILTER
    (RainToday=true) AS avgMinWithRain,
  Location FROM 'weatherAUS.csv.zst'
  GROUP BY ALL)

SELECT * FROM temp_rain WHERE
avgMin<avgMinWithRain
```



Aggregations

Standard aggregations

- DuckDB supports all of the SQL standard

Special add-on functions

- `any_val`
- `histogram`
- `list`
- `string_agg`

DuckDB additions

- `GROUP BY ALL`
groups by all non-selected values
- `ORDER BY` (in aggregation function)
relevant if order plays a role (like in `string_agg`)
- `GROUPING SETS`

Window Functions

Basic idea

- Perform calculations across a set of table row *related* to the current row
- Does not collapse the data set

Example

```
SELECT Location, Date, Rainfall, SUM(Rainfall) OVER  
(PARTITION BY Location) AS TotalRainfall,  
SUM(Rainfall) OVER (PARTITION BY location ORDER BY  
Date) AS TotalRainfallUntilToday, row_number() OVER  
(PARTITION BY location ) AS row_number FROM ...
```

Location	Date	Rainfall	TotalRainfall	row number
Adelaide	9/1/15	0.4	52.2	1
Adelaide	9/2/15	8.4	52.2	2
Adelaide	9/3/15	10.8	52.2	3
Adelaide	9/4/15	0	52.2	4
Adelaide	9/5/15	0	52.2	5
Adelaide	9/6/15	0	52.2	6
Adelaide	9/7/15	1	52.2	7
Adelaide	9/8/15	2.8	52.2	8
Adelaide	9/9/15	1.8	52.2	9
Adelaide	9/10/15	0	52.2	10

Location	Date	Rainfall	TotalRainfall	TotalRainfall- UntilToday	row number
Adelaide	2015-09-01	0.4	52.2	0.4	1
Adelaide	2015-09-04	0	52.2	19.6	4
Adelaide	2015-09-03	10.8	52.2	19.6	3
Adelaide	2015-09-05	0	52.2	19.6	5
Adelaide	2015-09-08	2.8	52.2	23.4	8
Adelaide	2015-09-02	8.4	52.2	8.8	2
Adelaide	2015-09-09	1.8	52.2	25.2	9
Adelaide	2015-09-06	0	52.2	19.6	6
Adelaide	2015-09-07	1	52.2	20.6	7
Adelaide	2015-09-10	0	52.2	25.2	10



Table creation

- CREATE OR REPLACE TABLE ...
- CREATE TABLE AS SELECT ...

Selection

- SELECT * EXCLUDE(field), COLUMNS()
- UN/PIVOT

Join types

- ASOF, LATERAL, SEMI, ANTI,
POSITIONAL

Advanced aggregation

- FILTER
- GROUPING SETS, GROUP BY CUBE

Functions and expressions

- Dot operator, e.g. .upper()
- List functions (like splits, comprehensions)
- Lambda functions
- Can be extended in Python (!)



Hands-on exercise:
Try complex queries on sample dataset



Q&A





Poll

Which features are supported by DuckDB's SQL dialect?

- Different kinds of JOINS
- Querying geographical data
- Common Table Expressions
- Window functions



DuckDB integration in Python



DuckDB lives in the same process as Python

Just one process

- If you instantiate duckdb from Python, they live in the same process and address space
- They can share memory and data

Tight integration

- No data must be copied
- Very important with large data volume

DataFrame integration with pandas

- Result of select can be converted to a DataFrame
- Can perform duckdb operations on DataFrame
- Same integration also works for polars



Different interaction possibilities

Query building

- Use builder objects and functions like `filter`, `aggregate`, `count` etc.
- Very Python-like, but needs learning
- Connection to SQL lost

Query with SQL

- Use DuckDB's SQL dialect directly
- Works like the CLI
- Easy to migrate to R etc.

DataFrame integration

- Convert result of DuckDB query to a `DataFrame`
- Perform duckdb operations on `DataFrame`
- Use familiar SQL idioms instead of (often opinionated) pandas functions
- Work independently of pandas or polars

Functions can be defined in Python

- No need to learn another language



Benefits of using duckdb in Python

In-process with zero setup



- Runs embedded inside the Python process itself. Import duckdb and start querying — no server, driver, or credentials to configure.

Standard DB-API 2.0



- Implements Python's standard `connect()/cursor()` interface, so it drops into SQLAlchemy and other existing tooling.

Zero-copy polars & pandas



- Query pandas and polars DataFrames directly with SQL through Apache Arrow, avoiding serialization overhead between engines.

Pythonic Relational API



- Chain `.filter()`, `.aggregate()`, and `.join()` calls as a fluent alternative to writing raw SQL strings.



Hands-on exercise: Run Jupyter notebook



Q&A





Poll

Which Python objects can you query with DuckDB?

- pandas DataFrames
- Python dicts
- polars DataFrames
- numpy arrays



DuckDB performance



Speed and optimal choices

Vectorized execution



- Operators process data in column batches (not row-by-row), using SIMD to cut per-row interpretation overhead

Multi-core parallelism



- Scans, joins, and aggregations are automatically split across all available CPU threads with no tuning required.

Columnar storage



- Data is stored by column, so analytical queries read only the columns they need — far less I/O than row stores.

Out-of-core processing



- Larger-than-RAM datasets spill to disk gracefully, so queries keep running instead of failing on memory limits.



Practical tuning

Set `memory_limit` and `threads` explicitly



- Defaults are conservative on shared machines. Give DuckDB most of the available RAM and cores for dedicated workloads.

Prefer Parquet over CSV and JSON



- Parquet keeps columnar layout and compression on disk, and lets DuckDB skip row groups that can't match a filter.

Profile with `EXPLAIN ANALYZE`



- Run it before optimizing blindly — it shows exactly which operator (scan, join, aggregate) is consuming the time.

Use filters and select minimal columns



- Filter and column pruning happen automatically for Parquet, but writing narrow queries makes pushdown reliable.



Tips from own practical experience

Extensions

- Useful also for performance optimization
- Use special functions (arg_min etc.)
- `SELECT * FROM duckdb_extensions();`

duckdb minimalism

- Only ingest what you need
- Use temporary tables
- Analyze data without persistence
- Use highly compressed parquet files

Performance Checklist

- ✓ Read fewer columns
- ✓ Filter early
- ✓ Prefer Parquet
- ✓ Leverage parallelism
- ✓ Profile with EXPLAIN ANALYZE
- ✓ Partition large datasets by frequently filtered columns.



Hands-on exercise:
**Run different queries and observe
performance**



Q&A





Real-life data analytics example

Real-life data analysis



Open data

- We need an open data source
- Data should be large and diverse
- *Not* the NYC taxi data

arXiv

- Largest preprint library
- Many different kinds of information
- Data is available at Kaggle
- Very large JSON file

arXiv is a free distribution service and an open-access archive for nearly 2.4 million scholarly articles in the fields of physics, mathematics, computer science, quantitative biology, quantitative finance, statistics, electrical engineering and systems science, and economics. Materials on this site are not peer-reviewed by arXiv.

Subject search and browse:
Physics Search Form Interface Catchup

Physics

- Astrophysics (astro-ph new, recent, search) Astrophysics of Galaxies; Cosmology and Nongalactic Astrophysics; Earth and Planetary Astrophysics; High Energy Astrophysical Phenomena; Instrumentation and Methods for Astrophysics; Solar and Stellar Astrophysics
- Condensed Matter (cond-mat new, recent, search) Disordered Systems and Neural Networks; Materials Science; Mesoscale and Nanoscale Physics; Other Condensed Matter; Quantum Gases; Soft Condensed Matter; Statistical Mechanics; Strongly Correlated Electrons; Superconductivity
- General Relativity and Quantum Cosmology (gr-qc new, recent, search)
- High Energy Physics - Experiment (hep-ex new, recent, search)
- High Energy Physics - Lattice (hep-lat new, recent, search)
- High Energy Physics - Phenomenology (hep-ph new, recent, search)
- High Energy Physics - Theory (hep-th new, recent, search)
- Mathematical Physics (math-ph new, recent, search)
- Nonlinear Sciences (nlin new, recent, search) Includes: Adaptation and Self-Organizing Systems; Cellular Automata and Lattice Gases; Chaotic Dynamics; Exactly Solvable and Integrable Systems; Pattern Formation and Solitons
- Nuclear Experiment (nucl-ex new, recent, search)
- Nuclear Theory (nucl-th new, recent, search)
- Physics (physics new, recent, search) Includes: Accelerator Physics; Applied Physics; Atmospheric and Oceanic Physics; Atomic and Molecular Clusters; Atomic Physics; Biological Physics; Chemical Physics; Classical Physics; Computational Physics; Data Analysis; Statistics and Probability; Fluid Dynamics; General Physics; Geophysics; History and Philosophy of Physics; Instrumentation and Detectors; Medical Physics; Optics; Physics and Society; Physics Education; Plasma Physics; Popular Physics; Space Physics
- Quantum Physics (quant-ph new, recent, search)

Mathematics

- Mathematics (math new, recent, search) Includes: (see detailed description): Algebraic Geometry; Algebraic Topology; Analysis of PDEs; Category Theory; Classical Analysis and ODEs; Combinatorics; Commutative Algebra; Complex Variables; Differential Geometry; Dynamical Systems; Functional Analysis; General Mathematics; General Topology; Geometric Topology; Group Theory; History and Overview; Information Theory; K-Theory and Homology; Logic; Mathematical Physics; Metric Geometry; Number Theory; Numerical Analysis; Operator Algebras; Optimization and Control; Probability; Quantum Algebra; Representation Theory; Rings and Algebras; Spectral Theory; Statistics Theory; Symplectic Geometry

Computer Science

- Computing Research Repository (CoRR new, recent, search) Includes: (see detailed description): Artificial Intelligence; Computation and Language; Computational Complexity; Computational Engineering, Finance, and Science; Computational Geometry; Computer Science and Game Theory; Computer Vision and Pattern Recognition; Computers and Society; Cryptography and Security; Data Structures and Algorithms; Databases; Digital Libraries; Discrete Mathematics; Distributed, Parallel, and Cluster Computing; Emerging Technologies; Formal Languages and Automata Theory; General Literature; Graphics; Hardware Architecture; Human-Computer Interaction; Information Retrieval; Information Theory; Logic in Computer Science; Machine Learning; Mathematical Software; Multiagent Systems; Multimedia; Networking and Internet Architecture; Neural and Evolutionary Computing; Numerical Analysis; Operating Systems; Other Computer Science; Performance; Programming Languages; Robotics; Social and Information Networks; Software Engineering; Sound; Symbolic Computation; Systems and Control



Hands-on exercise:
Perform the analysis as a hybrid
Jupyter-DuckDB project



Q&A





Summary & discussion





Group Discussion

- Any questions left?
- What would you like to achieve with this technology?



Not yet discussed

- Geospatial data
- Transactions
- Macros
- Checkpoints
- Many more – this is just an introduction!
- Please take a look at the books



Review Course Outcomes

By the end of this course, you will understand:

- How to integrate duckdb into your data analysis (with or without Python).
- How to work with different data formats and how to query or consolidate them.
- How to use duckdb for online analytical processing (OLAP).
- How to integrate duckdb with pandas and avoid bottlenecks.
- How to speed up your data-driven Jupyter notebooks.



Resources

GitHub repository

- <https://github.com/datanizing/oreilly-duckdb>

Books

- [DuckDB in Action](#)
- [Getting Started with DuckDB](#)
- [DuckDB: Up and Running](#)

Online resources

- <https://duckdb.org/>

Website

- DuckDB has excellent documentation itself: <https://duckdb.org/docs/>

The background of the image is a solid blue color with a subtle gradient. On the left side, there is a large, faint, light-blue circular graphic that resembles a stylized animal head, possibly a bear or a dog, facing right. The O'Reilly logo is centered in the image.

O'REILLY®